Table of Contents

موازی‌سازی ۱
معماری کامپیوتر ۳
محاسبات ناهمگون یا HETEROGENEOUS ۶
معماری heterogeneous یا نامتجانس (ناهمگون) ۷
قابلیت محاسباتی یا compute capabilitiy ۹
الگوی محاسباتی ناهمگون ۹
CUDA : یک پلتفرم برای محاسبات ناهمگون ۱۲

موازی‌سازی

امروزه، موازی سازی در همه‌جا وجود دارد، و برنامه‌نویسی موازی به عنوان یک مبحث جدایی ناپـــذیر در دنیای برنامه‌نویسان به شمار میآید. موازی‌سازی در چندین سطوح، مرتبط با معماری طراحی می باشـــد. به صورت کلی دو نوع موازی‌سازی وجود دارد:

- موازی‌سازی Task
- موازی‌سازی Data

موازی‌سازی Task وقتی کاربرد پیدا می‌کند که چندین task و یا چندین تابع وجود دارد که می‌توانند به صورت مستقل از همدیگر کار کنند. در واقع موازی‌سازی در مورد Task تلاش می‌کنـد کـه توابـع را بـر روی هسته‌های مختلف تقسیم کند.

موازی‌سازی Data وقتی کاربرد پیدا می‌کند که داده‌های زیادی وجود دارد که امکان پردازش همزمان بر روی آن‌ها وجود دارد. در واقع موازی‌سازی Data بر روی تقسیم داده‌ها بـر روی چنـدین هسـته تمرکـز دارد.

برنامه‌نویسی CUDA برای مسایلی که در آن‌ها امکان محاسـبات مـوازی بـر روی داده‌هـا وجـود دارد، بسیار مناسب به شمار میآید. عمده تمرکز این کتاب نحوه حل برنامه نویسی موازی بـر روی داده هـا بـا استفاده از برنامه‌نویسی CUDA می‌باشد. بسیاری از برنامه‌هایی که برای پردازش داده‌های بزرگ هستند می‌توانند از مدل data-parallel برای افزایش سرعت محاسبات خود بهره ببرند. پـردازش data-parallel داده‌ها را بر روی تردهای موازی نگاشت می‌کند.

اولین گام در طراحی برنامه data-parallel این است که داده‌ها را بر روی تردها پخش کنیم، که هر ترد وظیفه‌ی کار بر روی قسمتی از داده را بر عهده میگیرد. در حالت کلی، دو رویکرد برای تقسیم دیتا وجود دارد: تقسیم block ی و تقسیم cyclic . در تقسیم block ی، بسیاری از المان‌های متوالی از داده، به صورت دسته‌ای در کنار هم هستند. هر دسته بر روی یک ترد انتساب داده شده، و تردها در هر لحظه فقط یک دسته از این داده‌ها را پردازش می‌کنند. در تقسیم cyclic ،میزان داده کمتری در دسته‌ها قرار میگیرد، پس طبیعتا تعداد دسته‌ها بیشتر از روش block ی خواهد بود. تردهای همسایه، دسته‌های همسایگی را نیز دریافت می‌کنند و هر ترد می‌تواند بیش از یک دسته را پردازش کند. انتخاب یک دسته جدید برای پردازش بر روی ترد، موجب پریدن هر چند دسته که در ترد است می‌شود. تصویر زیر دو مثال از نحوه‌ی تقسیم دیتای یک بعدی را نمایش میدهد. در تقسیم block ی، هر ترد تنها یک قسمت از داده‌ها را برای پردازش برمیدارد، و در تقسیم cyclic ، هر ترد بیشتر از یک دسته را برای پردازش برمیدارد.(هر رنگ مرتبط با یک ترد است، پس در این مورد چهار ترد وجود دارد).

Block partition: each thread takes one data block

Cyclic partition: each thread takes two data blocks

در تصویر زیر سه مثال ساده از دیتای دو بعدی را نشان میدهد: تقسیم blockی در راستای محور y، تقسیم block ای در راستای هر دو محور، و تقسیم cyclic در راستای محور x . بقیه‌ی حالت‌های ممکن به عنوان تمرین به عهده خودتان گذاشته می‌شود.

Block partition on one dimension *Block partition on both dimensions* *Cyclic partition on one dimension*

معمولا، داده به صورت یک بعدی ذخیره می‌شود، حتی اگر یک دید چند بعدی از داده داشته باشیم، همچنان بر روی حافظه به صورت یک بعدی نگاشت می‌شود. مشخص کردن این که چطور داده بر روی تردها تقسیم شود معمولا وابسته به این است که به صورت فیزیکی چطور ذخیره شده است، همچنین

این که ترتیب تردها به چه صورت می‌باشد. این که چطور تردها را مدیریت می‌کند تاثیر زیادی بر روی عملکرد کلی برنامه خواهد داشت.

عملکرد برنامه معمولا حساس به block size هست. مشخص کردن بهترین میزان تقسیم در روش block ی و در روش cyclic به معماری کامپیوتر وابسته است. در این مورد در ادامه بیشتر یاد خواهیم گرفت.

معماری کامپیوتر

چندین روش برای طبقه‌بندی کردن معماری کامپیوترها وجود دارد. یکی از این روش‌ها که خیلی استفاده می‌شود طبقه‌بند فلین است، که معماری کامپیوتر را به چهار دسته مختلف تقسیم می‌کند، این طبقه بندی بر اساس نحوه‌ی جریان یافتن instructions و data بر روی هسته‌های سیستم عامل هست که شامل موارد زیر می‌شود:

- Single Instruction Single Data (SISD)
- Single Instruction Multiple Data (SIMD)
- Multiple Instruction Single Data (MISD)
- Multiple Instruction Multiple Data (MIMD)

معماری یک دستورالعمل یک داده اشاره به کامپیوترهای قدیمی دارد: یک معماری سریال. که تنها یک هسته در چنین سیستم عاملی وجود دارد. و در هر لحظه تنها یک جریان دستورالعمل قادر به اجراست، و عملیات‌ها بر روی یک جریان داده اعمال می‌شوند.

یک دستورالعمل چند داده اشاره به نوعی از معماری موازی دارد. در این معماری چندین هسته وجود دارد. همه‌ی هسته‌ها یک جریان دستورالعمل را در هر لحظه اجرا می‌کنند، و هر کدام بر روی جریان داده‌های مختلف اجرا می‌شوند. کامپیوترهای برداری معمولا از نوع SIMD هستند، و بیشتر کامپیوترهای مدرن یک معماری SIMD را به کار میگیرند. شاید بزرگترین مزیت SIMD این است که، در لحظه نوشتن کد بر روی CPU ، برنامه‌نویس می‌تواند به صورت سریال فکر کند ولی به پرفرمنس یک عملیات موازی دست پیدا کند چرا که کامپایلر جزییات پیاده‌سازی را برعهده خواهد گرفت.

چندین دستورالعمل یک داده به یک معماری غیر رایج اشاره دارد، که هر هسته بر روی یک جریان داده‌ی یکسان با چندین جریان دستورالعملی متفاوت عملیاتی را انجام میدهد.

چندین دستورالعمل چندین داده به معماری رایج موازی سازی اشاره دارد که در آن چندین هسته بر روی چندین جریان داده عملیاتی را انجام میدهد، و هر کدام به صورت مستقل دستورالعمل‌هایی را اجرا می‌کنند. بسیاری از معماری‌های MIMD یک کامپوننت زیرین SIMD را نیز در خود دارند.

در سطح معماری، پیشرفت‌های زیادی به منظور رسیدن به موفقیت‌های زیر صورت پذیرفته است:

- کاهش تاخیر
- افزایش پهنای باند
- افزایش توان عملیاتی

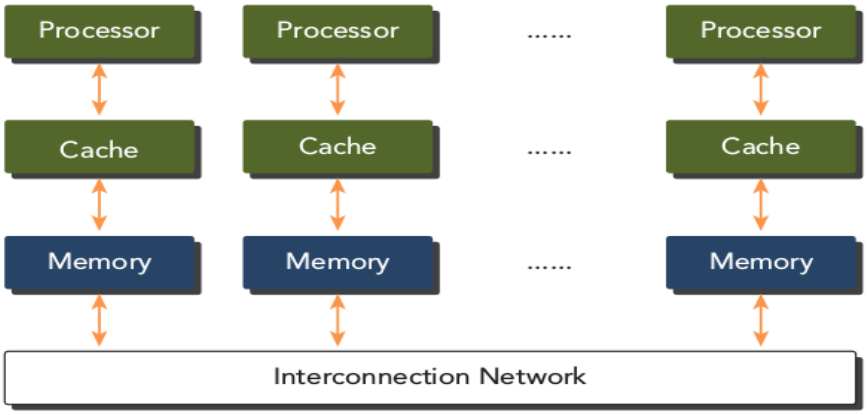
تاخیر یا latency به میزان زمانی اطلاق می‌شود که یک عملیات شروع و پایان می‌پذیرد، و معمولا در واحد میکروثانیه بیان می‌شود. **پهنای باند** مقدار داده‌ای است که می‌تواند در یک واحد زمانی پردازش شود، معمولا پهنای باند را تحت megabytes/sec و یا gigabytes/sec بیان می‌کنند. **توان عملیاتی** میزان دستوراتی است که می‌توان در یک واحد زمانی انجام داد، که معمولا تحت عنوان gflops بیان می‌شود(giga float operation per second) ، مخصوصا در فیلدهای علمی که محاسبات سنگین بر روی محاسبات ممیز شناور انجام می‌شود. تاخیر زمان سپری شده برای تکمیل شدن یک عملیات را نشان میدهد، در حالی که توان عملیاتی تعداد دستوراتی است که می‌توان در یک واحد زمانی انجام داد را نمایش میدهد.

معماری کامپیوتر همچنین می‌تواند بر اساس سازماندهی حافظه تقسیم شود، بر این مبنا کامپیوترها به دو دسته تقسیم می‌شوند.

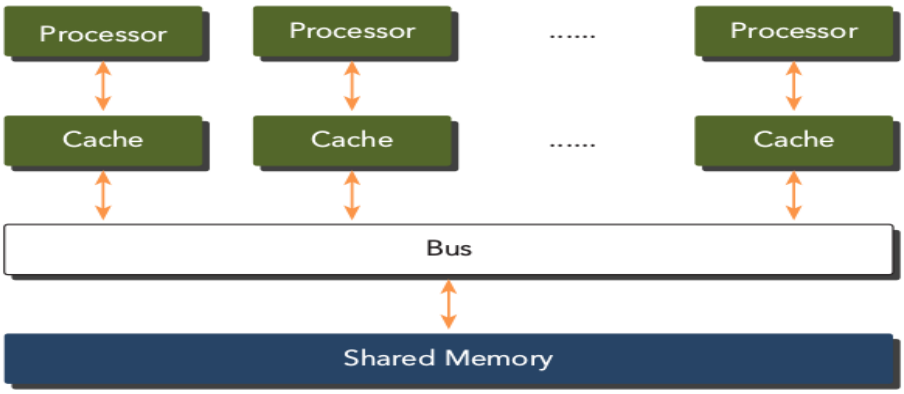
➤ Multi-node with distributed memory(چند نودی با حافظه توزیع شده)

➤ Multiprocessor with shared memory(چندپردازشگری با حافظه اشتراکی)

در یک سیستم چند نودی، موتورهای محاسباتی با ابعاد بزرگ ساخته شده که با شبکه به هم متصل هستند. هر پردازشگر حافظه محلی خود را دارد، و پردازشگرها می‌توانند محتوای حافظه محلی خود را بر روی شبکه به اشتراک بگذارند. شکل زیر یک سیستم چند نودی با حافظه توزیع شده را نمایش میدهد. چنین سیستم‌هایی را معمولا کلاستر می‌نامند.



معماری چند پردازنده‌ای (چند هسته‌ای) معمولا از رنج دو پردازنده‌ای(dual processor) تا هزاران یا صدها پردازنده می‌باشد. این پردازنده‌ها به صورت فیزیکی به یک حافظه مشترک متصل هستند، که در شکل بعدی مشخص می‌باشد، یا این که با یک لینک با تاخیر پایین(مثل PCI-Express یا PCIe) به هم متصل هستند.



اصطلاح many-core معمولا برای توصیف معماری چندهسته‌ای مخصوصا با تعداد هسته‌های بالا (ده ها یا صدها هسته) استفاده می‌شود. اخیرا، معماری کامپیوترها در حال انتقال از multi-core به many-core می‌باشد.

GPU ها یک معماری many-core دارند، و به صورت **مجازی** همه‌ی انواع موازی‌سازی که قبلا بیان شد را دارند: SIMD، MIMD، multi-threading و موازی سازی در سطح دستورات. NVIDIA از عبارت **single Instruction,Multiple thread(SIMT)** برای چنین معماری استفاده می‌کند.

CPU و GPU تاریخچه‌ی متفاوتی دارند. در واقع، GPU ها در گذشته به عنوان یک شتاب‌دهنده استفاده میشد. ولی اخیرا GPU ها خیلی قدرتمند، همه‌کاره، کاملا برنامه‌پذیر، قابلیت موازی‌سازی task و data را پیدا کرده‌اند و برای مسایلی که درگیر محاسبات موازی هستند بسیار ایده‌آل هستند.

تفاوت GPU و CPU در این دیدگاه

اگر چه ممکن است هم برای CPU و هم برای GPU از اصطلاح many-core استفاده شود ولی GPU از لحاظ معماری کاملا متفاوت از یک هسته CPU است.

یک هسته‌ی CPU، نسبتا سنگین وزن بوده، و برای محاسبات منطقی خیلی پیچیده طراحی شده است، هدف در این طراحی بهینه کردن محاسبات سریال و پشت سر هم می‌باشد.

یک هسته‌ی GPU ، نسبتا سبک وزن، و برای کارهای data-parallel با منطق ساده‌تر بهینه شده است، و تمرکز روی توان عملیاتی در برنامه‌های موازی است.

محاسبات ناهمگون یا HETEROGENEOUS

در روزهای آغازین دنیای کامپیوتر، کامپیوترها تنها شامل واحد پردازنده مرکزی بودند یعنی CPU ها که به منظور اجرای کارهای برنامه‌نویسی کلی طراحی شده بودند. ولی در دهه‌ی گذشته، کامپیوترهای پیشرو در محاسبات با کارآیی بالا، برخی المان‌های پردازشی دیگری را نیز استفاده کردند. شاخص‌ترین آن GPU هست، که ذاتا برای انجام محاسبات گرافیکی به صورت موازی طراحی شده است. در طی زمان، GPU ها قوی‌تر و جامع‌تر شده، و باعث شد که بتوان از آن‌ها برای مقاصد کلی در کارهای محاسباتی استفاده کرد یعنی کارهای general-purpose که این کار با یک کارآیی فوق العاده و توان مصرفی بهینه انجام می‌شود.

معمولا، CPU ها و GPU ها به عنوان واحدهای پردازشی مجزایی هستند که با باس PCI-Express و تنها یک نود محاسباتی به هم متصل می‌شوند. در این معماری، GPU ها را به عنوان یک دیوایس مجزا می‌بینند.

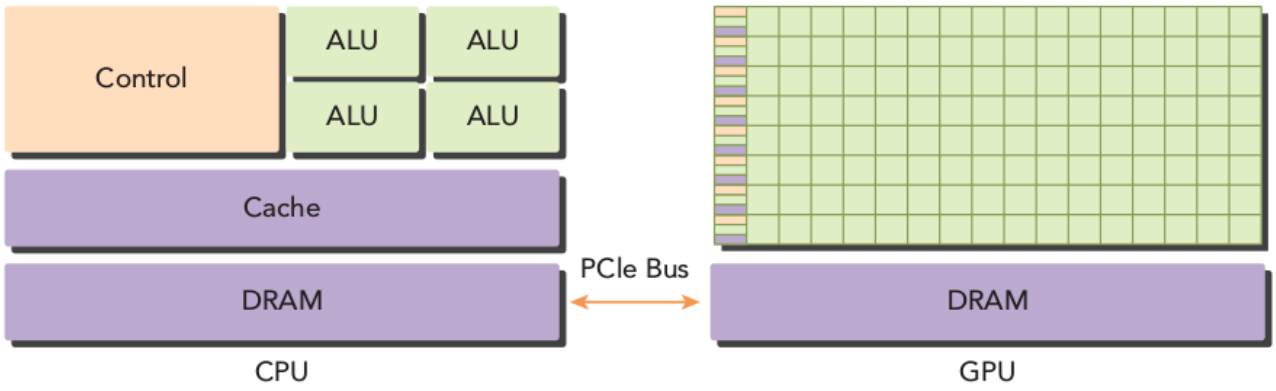
تغییراتی که منجر به استفاده از سیستم‌های ناهمگون به جای سیستم‌های همگون شد راهی بود که برای رسیدن به محاسبات با کارآیی بالا در تاریخ طی شد. سیستم‌های همگون از یک یا چندین پردازنده از یک معماری یکسان برای اجرای یک برنامه استفاده میکردند. سیستم‌های ناهمگون از معماری**های** مناسب برای اجرای برنامه استفاده می‌کنند، و هر کار متناسب با معماری به پردازنده تخصیص داده می‌شود، که منجر به افزایش کارآیی در این گونه سیستم‌ها می‌شود.

اگر چه سیستم‌های ناهمگون مزیت‌های خیلی مهمی نسبت به سیستم‌های قدیمی در محاسبات دارند، ولی امروزه استفاده موثر از این سیستم‌ها با افزایش پیچیدگی طراحی برنامه‌ها محدود شده است. در حالی که برنامه‌نویسی موازی توجه‌ها را به خود جلب کرده است، همچنان پیچیدگی‌هایی نیز در خود گنجانده است.

اگر شما در زمینه‌ی برنامه‌نویسی موازی تازه‌کار هستید، در این صورت می‌توانید از ابزارهای پیشرفته و توسعه‌هایی که در زمینه‌ی سیستم های ناهمگون رخ داده است استفاده لازم را ببرید. اگر که یک برنامه‌نویس سطح بالا در زمینه‌ی پردازش موازی هستید، تطابق یافتن با معماری ناهمگون برای شما آسان‌تر خواهد بود.

معماری heterogeneous یا نامتجانس (ناهمگون)

امروزه، نودهای محاسباتی در یک معماری عادی ناهمگون متشکل از دو سوکت CPU و دو یا بیشتر GPU است. یک GPU دیگر یک پلتفرم مستقل نیست بلکه یک پردازنده کمکی برای CPU می‌باشد. بنابراین، GPU ها باید به عنوان یک دیوایس برای CPU با استفاده از یک باس PCI-Express باشند، که در شکل زیر نمایش داده شده است. دقیقا به همین دلیل است که، در اصطلاح برنامه‌نویسی GPU ما از CPU به عنوان host و از GPU به عنوان device استفاده می‌کنیم.



در heterogeneous ها برنامه از دو بخش تشکیل میگردد.

- کد میزبان (Host code)
- کد دیوایس (Device code)

Host code بر روی CPU ها اجرا شده و device code بر روی GPU ها اجرا می‌شود. یک برنامه که بر روی یک پلتفرم heterogeous اجرا می‌شود، معمولا با CPU شروع می‌شود. کد CPU مسوولیت‌های روبه‌رو را دارد: مدیریت محیط، کد، و داده‌ای است که دیوایس میگیرد.

با برنامه‌های محاسباتی سنگین، معمولا program section مقدار قابل توجهی از داده‌های موازی را در خود خواهد داشت. GPU ها برای سرعت بخشیدن به اجرای این قسمت از داده‌ی موازی هستند. وقتی که یک قطعه‌ی سخت‌افزاری که به صورت سخت‌افزاری نیز از CPU جدا می‌باشد به منظور سرعت بخشیدن به این قسمت‌ها استفاده شود، معمولا از آن به عنوان یک شتاب دهنده یاد می‌شود. GPU ها به عنوان بارزترین شتابدهنده‌ی سخت‌افزاری می‌باشد.

پلتفرم محاسباتی NVIDA's GPU در سری‌های زیر فعال هستند:

- Tegra
- GeForce
- Quadro
- Tesla

خانواده‌ی Tegra به منظور استفاده در موبایل‌ها و ابزارهای نهفته مانند تبلت‌ها و گوشی‌ها طراحی شده، Gefore برای استفاده کاربران، Quadro برای مصورسازی‌های حرفه‌ای و Tesla برای دیتاسنترهای محاسبات موازی هستند. Fermi، شتاب‌دهنده‌ی GPU از خانواده‌ی Tesla، اخیرا توجهات زیادی را به عنوان شتاب‌دهنده‌ی محاسباتی در برنامه‌های با کارآیی بالا جلب کرده است. Fermi، توسط NVIDIA در سال ۲۰۱۰ به عنوان اولین معماری کامل محاسباتی GPU رونمایی شد. GPU شتاب‌دهنده‌ی Fermi تا به امروز در بسیاری از زمینه‌های محاسباتی استفاده شده است مثل مطالعات زمین‌لرزه، شبیه‌سازی‌های بیوشیمی، مدل‌سازی آب و هوا، و پردازش سیگنال، محاسبات مالی، آنالیز داده. Kepler، نسل فعلی معماری GPU که بعد از Fermi بود، در پاییز ۲۰۱۲ رونمایی شد، که قدرت پردازشی خیلی بیشتری را نسبت به نسل قبلی پیشنهاد می‌کرد و روش‌های جدیدی برای بهبود و بهینه کردن اجراهای موازی در اختیار قرار میداد. Tegra K1 از Kepler به منظور افزایش قدرت محصولات نهفته استفاده کرده است.

ویژگی‌های مهمی وجود دارد که مشخص کننده‌ی توانایی GPU می‌باشد:

- تعداد هسته‌های CUDA
- میزان حافظه

همچنین، دو متریک برای اندازه‌گیری کارآیی یک GPU وجود دارد:

- peak computational performance
- memory bandwidth

peak computational performance راهی برای اندازه‌گیری قابلیت محاسباتی است، که به این صورت تعریف می‌شود که چقدر محاسبات ممیز شناور single یا double را می‌توان در یک ثانیه انجام داد. Peak performance معمولا به صورت gflops و یا tflops بیان می‌شود. پهنای باند حافظه با gigabytes/sec بیان می‌شود. جدول زیر یک خلاصه‌ای از معماری Fermi و Kepler و ویژگی‌های کارآیی را آورده است.

	FERMI (TESLA C2050)	KEPLER (TESLA K10)
CUDA Cores	448	2 x 1536
Memory	6 GB	8 GB
Peak Performance*	1.03 Tflops	4.58 Tflops
Memory Bandwidth	144 GB/s	320 GB/s

* Peak single-precision floating point performance

بیشتر مثال‌هایی که در این کتاب زده می‌شود هم بر روی Fermi قابل اجرا خواهد بود و هم بر روی Kepler. برخی از مثال‌ها نیازمند یک معماری خاص می‌باشد که تنها در GPU های Kepler گنجانده شده است.

قابلیت محاسباتی یا compute capabilitiy

NVIDIA از یک اصطلاح خاص یعنی compute capability برای ورژن سخت‌افزار GPU برای همه‌ی خانواده‌ی Tesla استفاده می‌کند. دیوایس‌ها با ورژن کلی یکسان معماری هسته یکسانی خواهند داشت.

- معماری کلاس Kepler با عدد ۳ نمایش داده می‌شود.
- معماری کلاس Fermi با عدد ۲ نمایش داده می‌شود.
- معماری کلاس Tesla با عدد ۱ نمایش داده می‌شود.

اولین کلاس GPU هایی که توسط NVIDIA رونمایی شد، در خود نام Tesla را برای همه‌ی GPU ها نیز داشت (یعنی هم نام Tesla بود و هم معماری سخت‌افزار هسته Tesla بود). همه‌ی مثال‌های این کتاب نیاز به cp بالای ۲ خواهند داشت.

TABLE 1-2: Compute Capabilities of Tesla GPU Computing Products

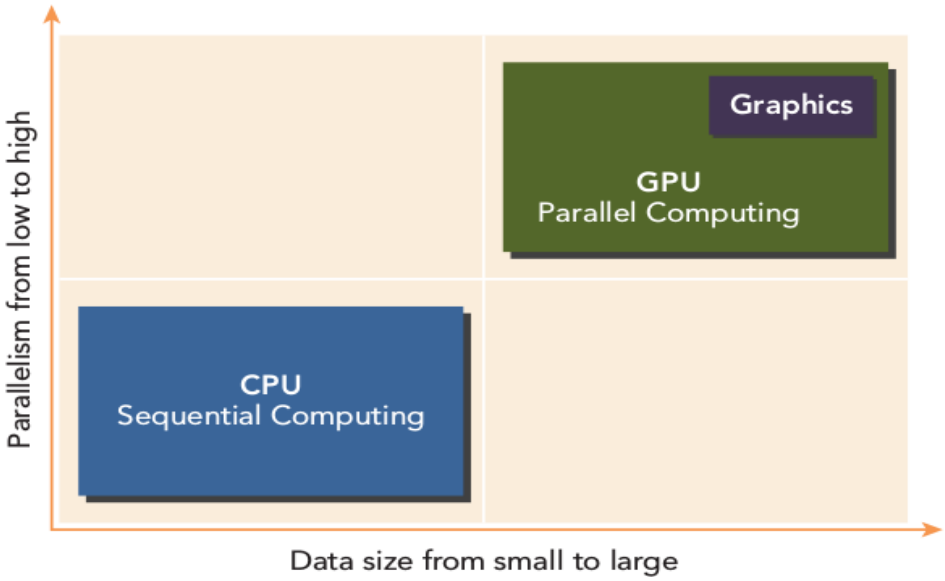
GPU	COMPUTE CAPABILITY
Tesla K40	3.5
Tesla K20	3.5
Tesla K10	3.0
Tesla C2070	2.0
Tesla C1060	1.3

الگوی محاسباتی ناهمگون

محاسبات GPU قصد جایگزینی با محاسبات CPU را ندارند. هر کدام مزیت‌های خاصی برای نوع خاصی از برنامه‌ها دارند. محاسبات CPU برای عملیات‌هایی که کنترل سختی دارند مناسب می‌باشد، و GPU برای محاسباتی مفید هستند که در آن Task ها محاسبات موازی زیادی دارند. وقتی CPU همراه با GPU قرار میگیرد، یک ترکیب خیلی قوی را ایجاد خواهند کرد. CPU برای محاسبات کوتاه که کنترل غیر قابل پیش‌بینی دارند مناسب می‌باشد، و GPU برای محاسبات سنگین ولی کنترل سبک مناسب می‌باشد. در واقع دو پارامتر برای انتخاب بهتر بین CPU و GPU وجود دارد:

- سطح موازی سازی
- اندازه داده

اگر یک مساله حجم داده کمی داشته باشد، و منطق برنامه پیچیده باشد، و یا سطح موازی‌سازی پایینی داشته باشد، در این صورت CPU انتخاب مناسب‌تری می‌باشد چرا که توانایی مناسبی در منطق پیچیده‌تر دارد. اگر یک برنامه دارای مقدار زیادی داده بود، و سطح موازی‌سازی بالایی داشته باشد، در این صورت GPU انتخاب مناسب تری می‌باشد چرا که GPU تعداد هسته‌های بسیار بیشتری دارد، که می‌تواند از multi-threading به صورت گسترده استفاده کند، و نسب به CPU دارای عرض پهنای باند بیشتری می‌باشد.



معماری محاسبات موازی ناهمگون CPU+GPU به این دلیل به وجود آمد که CPU و GPU هر کدام ویژگی‌های مکملی دارند که باعث می‌شود برنامه‌ها برای هر چیزی مهیا باشند. بنابراین، به منظور کارآیی بهینه ممکن است که نیاز داشته باشید برای برنامه خودتان همه از CPU و هم از GPU استفاده کنید، اجرای قسمت ترتیبی و یا قسمت موازی بر روی CPU و قسمتی که دارای داده‌ی موازی سنگین هست بر روی GPU که در شکل زیر نمایش داده شده است.

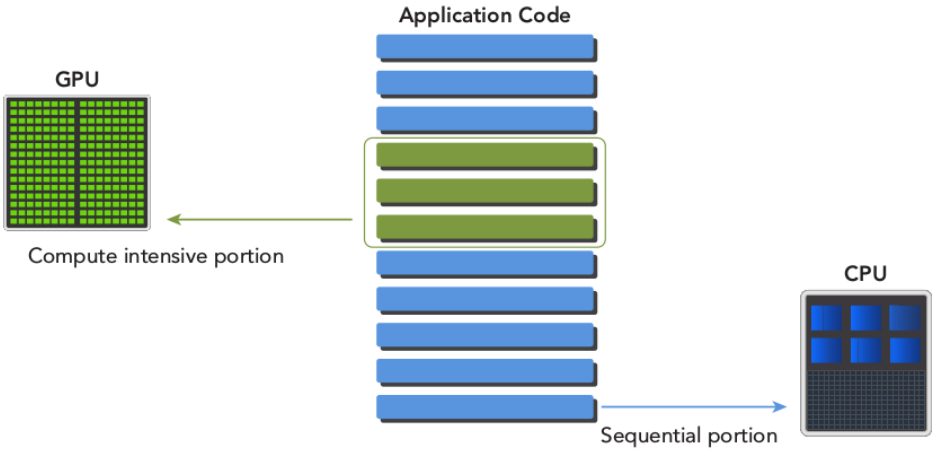


FIGURE 1-11

نوشتن کد به این صورت به شما این اطمینان را میدهد که ویژگی‌های GPU و CPU مکمل همدیگر باشند، و منجر به استفاده‌ی حداکثری از قدرت CPU+GPU خواهد شد. برای این که بتونیم یک برنامه طراحی کنیم که از ترکیب CPU+GPU پشتیبانی داشته باشد، NVIDIA یک مدل برنامه‌نویسی به نام CUDA ایجاد کرده است. این مدل برنامه‌نویسی جدید تمرکز کتاب در ادامه خواهد بود.

تفاوت ترد CPU با ترد GPU

تردها روی CPU به عنوان موجودیت‌های سنگینی شناخته می‌شوند. سیستم عامل باید یک ترد را خاموش و روشن کند تا توانایی multi-threading را فعال کند. Context switches کند و پر هزینه هستند.

تردها بر روی GPU خیلی سبک هستند. در یک سیستم عادی، ده ها هزار ترد در صف هستند تا کاری را انجام دهند. اگر یک GPU باید منتظر گروهی از تردها باشد، خیلی راحت می‌تواند یک کار دیگر را اجرا کند.

هسته‌های CPU طراحی شده‌اند تا میزان latency را کمینه کنند، در حالی که هسته‌های GPU برای تعداد زیادی ترد موازی طراحی شده‌اند تا قابلیت محاسباتی را بیشینه کنند.

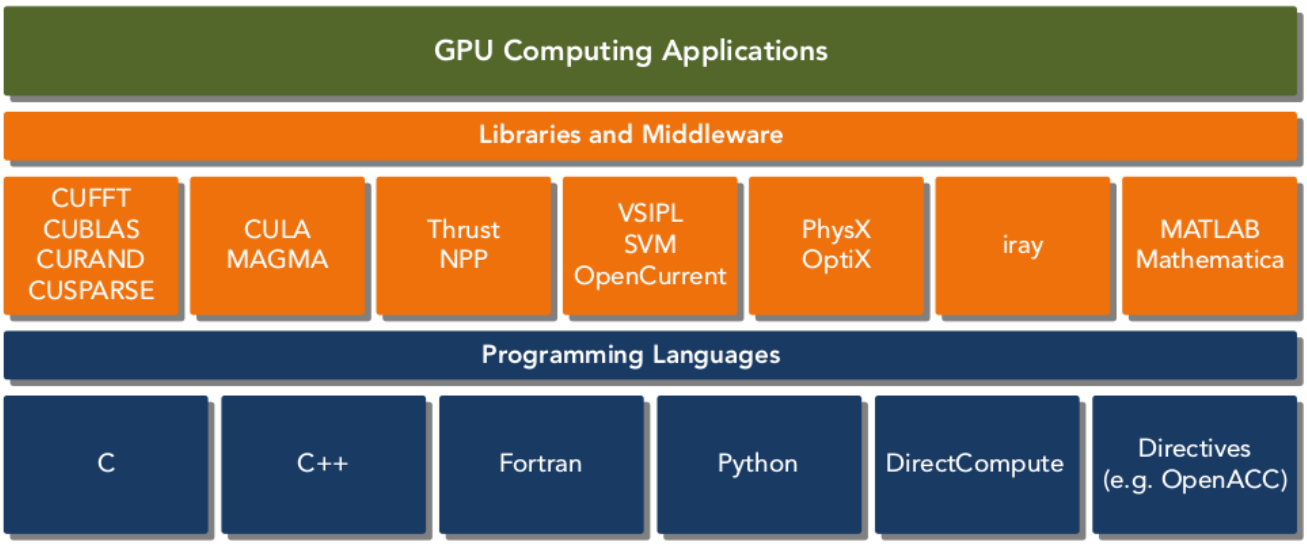
امروزه، یک CPU با چهار هسته quad می‌تواند تنها ۱۶ تا ترد را به صورت همزمان اجرا کند، البته اگر CPU بتواند از hyper-threading پشتیبانی کند می‌توان آن را تا ۳۲ تا ترد افزایش داد.

GPU های مدرن NVIDIA می‌توانند تا ۱۵۳۶ تا ترد فعال به ازای هر multiprocessor داشته باشند. روی یک GPU با ۱۶ تا multiprocessor، این عدد تا ۲۴۰۰۰ ترد فعال افزایش می‌یابد.

CUDA : یک پلتفرم برای محاسبات ناهمگون

CUDA یک پلتفرم محاسبات موازی general-purpose می‌باشد، یک مدل برنامه‌نویسی که موتورهای برنامه‌نویسی موازی در GPU های NVIDIA را به منظور حل مسایل پیچیده به صورت موثری به کار میگیرد. با استفاده از CUDA ، شما می‌توانید به GPU به منظور محاسبات دسترسی داشته باشید، که قبلا بر روی CPU اجرا میشد.

پلتفرم CUDA از طریق کتابخانه‌های شتابدهنده‌ی CUDA ، رابط برنامه‌نویسی و افزونه‌هایی که برای زبان‌های استاندارد فراهم شده شامل C,C++,Fortran و python قابل دسترسی خواهد بود. این کتاب بر روی CUDA C programming تمرکز خواهد داشت.



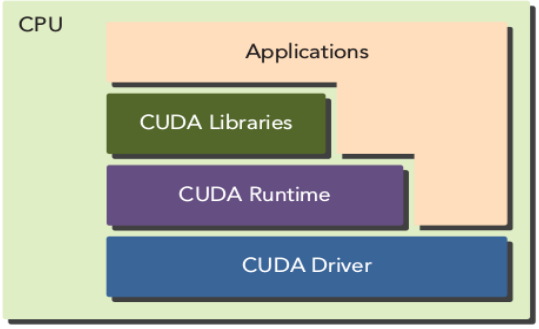
CUDA C یک افزونه از ANSI C استاندارد می‌باشد که افزونه‌های زبان به درد بخوری برای فعال کردن برنامه‌نویسی ناهمگون در اختیار ما قرار میدهد، همچنین یک API سر راست برای مدیریت device ، حافظه، و task های دیگر نیز دارد. CUDA همچنین یک مدل برنامه‌نویسی scalable دارد که می‌شود میزان scalable بودن موازی سازی را با تغییر تعداد هسته‌ها تغییر داد، در عین حال نیازی به معلومات پیچیده در مورد GPU نخواهد بود و برای کسی که زبان C را یاد گرفته این کار آسان خواهد بود.

CUDA دو سطح API برای مدیریت GPU device و مدیریت تردها فراهم کرده است که بعدا در شکل نیز خواهیم دید:

- CUDA Driver API
- CUDA Runtime API

API مربوط به driver یک API سطح پایین بوده و تقریبا برای برنامه‌نویسی دشوار می‌باشد، اما کنترل بیشتری بر روی GPU در اختیار ما قرار میدهد. API مربوط به runtime یک API سطح بالا بوده که بر

روی API درایور ساخته شده است. هر تابع از API مربوط به runtime به عملیات‌های ابتدایی تر که مربوط به API درایور است شکسته می‌شود.



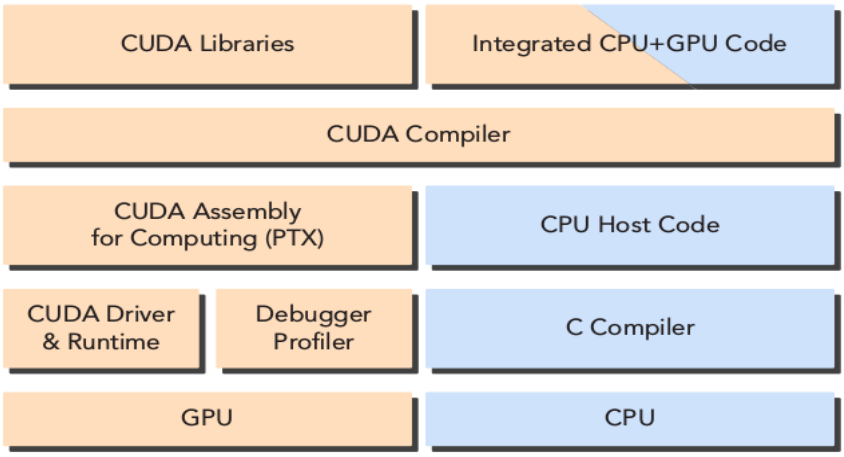
در حقیقت در مورد کارآیی این دو API تفاوت زیادی وجود ندارد. این که چطور کرنل شما از حافظه استفاده می‌کند و چطور تردها را مدیریت می‌کنید چیزی است که تفاوت کارآیی را مشخص می‌کند.

این دو API دو به دو با همدیگر ناسازگار هستند. شما باید یکی از این دو را انتخاب کنید، و این امکان وجود ندارد که توابع این دو را با هم ترکیب کنید. همه‌ی مثال‌ها در این کتاب از runtime استفاده می‌کنند.

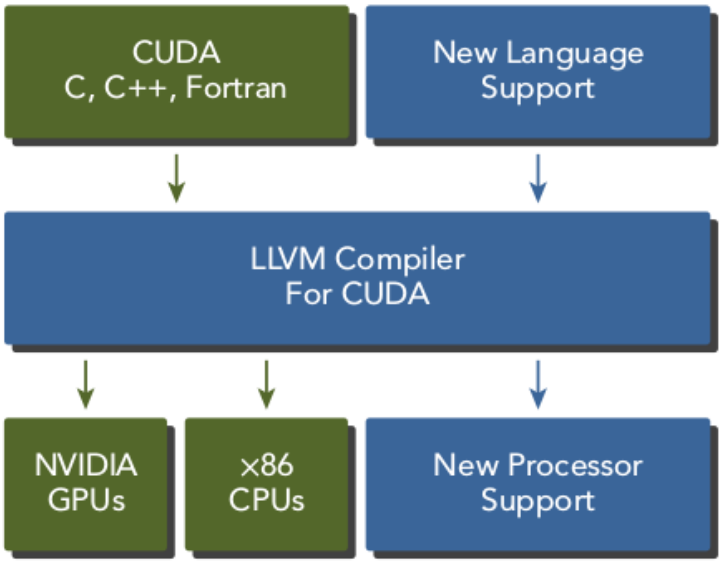
یک برنامه CUDA ترکیبی از دو بخش زیر می‌باشد.

- یک کد میزبان که بر روی CPU اجرا می‌شود.
- یک کد device که بر روی GPU اجرا می‌شود.

کامپایلر nvcc مربوط به کودای NVIDIA در طی مرحله کامپایل کد device را از کد میزان جدا می‌کند. Host code در واقع زبان C استاندارد بوده و در ادامه توسط کامپایلر زبان C کامپایل خواهد شد. کد device با استفاده از کلمات کلیدی برای مشخص کردن توابع data-parallel جدا می‌شود که kernel نامیده می‌شود. در ادامه کد device بر روی nvcc کامپایل می‌شود. در مرحله link، کتابخانه‌ی CUDA runtime به فراخوان‌های kernel اضافه شده و به طور صریح بر روی GPU می‌تواند اجرا شود.



کامپایلر nvcc عمدتا بر اساس ;کامپایلر LLVM نوشته شده است. شما می‌توانید با اســتفاده از CUDA SDK Compiler یک زبان برنامه‌نویسی بسازید و یا توسعه دهید که از شــتابدهنده‌ی GPU پشــتیبانی کند.



The CUDA platform is also a foundation that supports a diverse parallel computing ecosystem, as shown in Figure 1-16. Today, the CUDA ecosystem is growing rapidly as more and more companies provide world-class tools, services, and solutions. If you want to build your applications on GPUs,the easiest way to harness the performance of GPUs is with the CUDA Toolkit (https://developer.nvidia.com/cuda-toolkit), which provides a comprehensive development environment for C and C++ developers. The CUDA Toolkit includes a compiler, math libraries, and tools for debugging and optimizing the performance of your applications. You will also fi nd code samples,programming guides, user manuals, API references, and other documentation to help you get started.

اولین کدمان را به صورت زیر می‌نویسیم.

```
#include <stdio.h>
#include <iostream>
using namespace std;
__global__ void hellofromGPU(void)
{
    printf("Hello world from GPU!! \n");
}
int main()
```

```
{
    cout<<"Hello world from CPU!!"<<endl;
    hellofromGPU<<<1,1>>>();
    cudaDeviceReset();
    return 0;
}
```

در این کد ما تابع hellofromGPU را به صورت زیر تعریف کرده‌ایم.

```
__global__ void hellofromGPU(void)
{
    printf("Hello world from GPU!! \n");
}
```

این تابع یک تابع کرنل است که در آن ___global___ یک qualifier به حساب می‌آید. این qualifier به کامپایلر می‌گوید که این تابع از روی CPU خوانده می‌شود و روی GPU اجرا می‌شود. برای اجرا کردن تابع کرنل باید به صورت زیر عمل کنیم.

```
hellofromGPU<<<1,10>>>();
```

در این کد triple angle bracket یک فراخوانی از ترد میزبان به کدی که بر روی device قرار دارد مارک می‌کند. یک کرنل توسط آرایه‌ای از تردها اجرا شده و همه‌ی تردها یک کد یکسان را اجرا خواهند کرد. پارامترهایی که درون براکت سه‌گانه آورده شده است، پارامترهای تنظیمی برای اجرا می‌باشد، که مشخص کننده‌ی تعداد تردهایی است که تابع کرنل را اجرا می‌کنند. در این مثال، شما از ۱۰ ترد برای اجرای کرنل استفاده می‌کنید. و خروجی به صورت زیر خواهد بود.

```
Hello world from CPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
Hello world from GPU!!
```

تابع cudaDeviceReset() صریحا همه‌ی منابعی که این پردازش از device گرفته را تخریب و پاک می‌کند. برای کامپایل بر روی معماری sm_20 به صورت زیر نیز می‌توانیم عمل کنیم.

```
nvcc -arch sm_20 hello.cu -o hello
```

یک برنامه‌ی عادی کودا شامل پنج قدم خواهد بود.

۱. اختصاص دادن حافظه‌ی GPU
۲. کپی کردن داده از روی حافظه‌ی CPU روی حافظه‌ی GPU.
۳. فراخوانی کردن یک تابع کرنل کودا برای انجام یک محاسبه‌ی خاص.
۴. کپی کردن داده از حافظه‌ی GPU روی حافظه‌ی CPU.
۵. نابود کردن حافظه‌ای که به GPU اختصاص داده بودیم.

A typical CUDA program structure consists of five main steps:
1. Allocate GPU memories.
2. Copy data from CPU memory to GPU memory.
3. Invoke the CUDA kernel to perform program-specific computation.
4. Copy data back from GPU memory to CPU memory.
5. Destroy GPU memories.
In the simple program hello.cu, you only see the third step: Invoke the kernel. For the remainder of this book, examples will demonstrate each step in the CUDA program structure .